

Mini Project Report

on

California Housing Price Prediction

Submitted By
Anshul Narendra Parate

Course Name: Machine Learning



Department of Computer Science and Engineering
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Shri Ramdeobaba College of Engineering & Management, Nagpur 440013

(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University Nagpur)

October 2025

1. Problem Statement

In the modern real estate market, particularly within the state of California, housing prices are influenced by a complex interplay of demographic, economic, and geographical factors. With rapid urbanization, fluctuating economic conditions, and growing demand for residential spaces, accurate prediction of housing prices has become an essential tool for investors, policymakers, and potential homeowners.

The primary challenge in understanding housing market dynamics lies in the non-linear relationships between various independent factors—such as median income, population density, proximity to the ocean, and average number of rooms—and the target variable, median house value. Traditional statistical methods often fail to capture these intricate dependencies effectively, leading to suboptimal predictions.

Machine learning offers a data-driven approach to address this challenge. By analyzing historical housing data and learning from patterns across multiple features, machine learning models can accurately estimate property prices and identify the most influential factors affecting valuation.

This project applies the principles of predictive analytics to the California Housing Dataset, aiming to develop a regression-based model that predicts the median housing price of a district based on its socio-economic and geographic characteristics. The ultimate goal is to build an interpretable, high-performance predictive system that can assist in decision-making processes for real estate developers, analysts, and policy planners.

The project's contribution lies in designing and implementing an end-to-end predictive solution — from data preprocessing and exploratory analysis to model selection, optimization, and deployment. This solution transitions housing valuation from a subjective estimation process to a reliable, data-informed prediction model, thereby improving accuracy, transparency, and efficiency in real estate decision-making.

2. Project Objectives

To develop an accurate and efficient predictive model for California housing prices, the project was divided into four systematic and sequential objectives:

Comprehensive Data Preprocessing and Feature Engineering:

The foundation of this project lies in a detailed exploration and preparation of the California Housing Dataset. This involves handling missing values, detecting outliers, and transforming features to improve model interpretability and performance. Categorical variables (if any) are encoded using appropriate techniques such as One-Hot Encoding, while numerical features like *median_income* and *households* are scaled for uniformity. New derived features—such as *rooms_per_household* or *population_per_household*—are engineered to capture underlying relationships in the data. This step ensures the dataset is clean, consistent, and ready for modeling.

Feature Correlation and Multicollinearity Analysis:

Since housing prices depend on multiple correlated factors, it is essential to identify redundant or highly correlated features that could distort the learning process. This objective involves performing statistical correlation analysis and using tools like heatmaps and Variance Inflation Factor (VIF) to retain only the most significant predictors. By reducing multicollinearity, the model's stability and interpretability are enhanced.

Systematic Model Evaluation and Selection:

This phase involves training a diverse set of regression algorithms—including Linear Regression, Decision Tree Regressor, Random Forest Regressor, and XGBoost Regressor—on the processed dataset. Each model is evaluated on an unseen test set using performance metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score. These metrics ensure a balanced evaluation between model accuracy and generalization. The best-performing model is selected based on its ability to minimize error and maximize explanatory power.

Model Optimization and Practical Deployment:

The final objective focuses on enhancing and deploying the selected model for real-world usability. GridSearchCV is employed to perform hyperparameter tuning, systematically identifying the optimal configuration that yields the lowest RMSE and highest R^2 score. The optimized model is then serialized using Joblib or Pickle and integrated into an interactive web interface built with Streamlit, allowing users to input housing parameters and instantly obtain predicted price values. This deployment bridges the gap between complex machine learning analysis and accessible, practical decision support for end-users such as real estate agents, policymakers, and data analysts.

3. Methodology

The project was executed following a structured and systematic workflow that encompasses data collection, preprocessing, model development, evaluation, and deployment. Each phase contributes to building a reliable and interpretable regression-based prediction system.

3.1. Data Collection and Preparation

The project utilizes the California Housing Dataset, originally derived from the 1990 U.S. Census data. It contains information about various demographic, economic, and geographic attributes of housing districts in California.

The features are categorized as follows:

- Geographical Features: `longitude`, `latitude`
- Housing Characteristics: `housingMedianAge`, `totalRooms`, `totalBedrooms`, `households`
- Population and Income: `population`, `medianIncome`
- Proximity Indicator: `oceanProximity` (categorical variable)
- Target Variable: `medianHouseValue` (continuous numeric value in U.S. dollars)

This dataset serves as the foundation for developing predictive regression models aimed at estimating housing prices based on multiple interrelated attributes.

3.2. Data Preprocessing

1. Handling Missing Values:

The dataset was examined for null or missing entries. For numerical columns such as `totalBedrooms`, missing values (if any) were handled using mean imputation, ensuring that model training is not disrupted.

2. Feature Engineering:

To enhance predictive power, new derived features were created, including:

- `rooms_per_household = totalRooms / households`
- `population_per_household = population / households`
- `bedrooms_per_room = totalBedrooms / totalRooms`

These engineered features provide deeper insights into living conditions and density patterns that influence housing prices.

3. Encoding Categorical Data:

The **oceanProximity** feature was transformed using One-Hot Encoding to convert categorical values (e.g., "<1H OCEAN", "INLAND", "NEAR BAY") into binary columns. This ensures compatibility with regression algorithms that require numerical input.

4. Feature Scaling:

Continuous features were standardized using StandardScaler to ensure uniform data distribution. This step helps models like Linear Regression and XGBoost converge more efficiently and maintain numerical stability.

3.3. Train-Test Split

The processed dataset was split into two subsets:

- Training Set: 80% of the data, used for model fitting.
- Testing Set: 20% of the data, used for performance evaluation.
This division ensures that model accuracy and generalization are evaluated on unseen data.

3.4. Model Training, Selection, and Tuning

1. Baseline Model Comparison:

Several regression algorithms were trained and compared, including:

- Linear Regression (baseline model)
- Multiple Linear Regression
- Random Forest Regressor
- XGBoost Regressor
- LightGBM Regressor

2. Each model was evaluated using metrics such as R^2 Score, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE) to assess both accuracy and robustness.

3. Model Evaluation and Selection:

Ensemble models, particularly Random Forest, XGBoost, and LightGBM, demonstrated superior predictive performance compared to linear models due to their ability to capture non-linear relationships and feature interactions.

4. Hyperparameter Tuning:

For further optimization, GridSearchCV was applied to ensemble models (XGBoost and LightGBM) to systematically identify the best combination of parameters—such as tree depth, learning rate, and number of estimators—that maximize R^2 Score and minimize prediction error.

5. Final Model Deployment:

The tuned LightGBM Regressor, achieving the highest accuracy and lowest RMSE, was selected as the final model. It was serialized using Joblib and deployed via a Streamlit web application, allowing real-time prediction of housing prices based on user-input features.

4. Technology Stack

This project employs a comprehensive stack of Python libraries and frameworks, covering every stage of the workflow—from data preprocessing and model development to evaluation and deployment. The specific components used are outlined below.

Core Data Science & Utility Libraries

- Python 3.x: The primary programming language used for data handling, model building, and deployment.
- Pandas: Utilized for data manipulation and analysis using DataFrames (`pd.read_csv`, `df.describe`, `df.info`). It enabled efficient handling of large tabular data.
- NumPy: Provided support for high-performance numerical computations and array operations.
- SciPy: Used for statistical analysis, data distribution tests, and mathematical operations.
- Standard Library Modules:
 - `os` for directory and file path management.
 - `math` for numerical computations.
 - `warnings` configured with `filterwarnings('ignore')` to suppress non-critical runtime messages.
- IPython: Employed for enhanced visualization and interactive execution within Jupyter Notebook environments.

Data Visualization

- Matplotlib: The base library for creating static, animated, and publication-quality plots (`plt.plot`, `plt.hist`, etc.).
- Seaborn: Built on Matplotlib, it was used for statistical and exploratory data visualization (`sns.pairplot`, `sns.heatmap`, etc.) to identify relationships and correlations between features.
- Plotly (optional): Used for interactive visualizations, enabling detailed inspection of variable trends and model outputs.

Preprocessing, Pipelines, and Feature Engineering

- Scikit-learn (sklearn.preprocessing):
 - **StandardScaler** for feature scaling to normalize the range of continuous variables.
 - **OneHotEncoder** for converting the categorical feature **oceanProximity** into numerical form.
- Scikit-learn (sklearn.impute):
 - **SimpleImputer** used to handle missing values in attributes like **totalBedrooms**.
- Scikit-learn (sklearn.compose & sklearn.pipeline):
 - **ColumnTransformer** applied different preprocessing strategies for numerical and categorical data.
 - **Pipeline** linked preprocessing and modeling steps, reducing redundancy and preventing data leakage.

Machine Learning Models

- Scikit-learn (sklearn.linear_model):
 - **LinearRegression** and **Ridge/Lasso** Regression models served as baselines for performance comparison.
- Scikit-learn (sklearn.tree):
 - **DecisionTreeRegressor** tested for non-linear relationships in housing data.
- Scikit-learn (sklearn.ensemble):
 - **RandomForestRegressor** implemented for robust ensemble learning and high-variance reduction.
- XGBoost (xgboost):
 - Used for Extreme Gradient Boosting, offering superior accuracy and fast performance on structured data.

- LightGBM (lightgbm):
 - Applied as a gradient boosting framework optimized for large datasets and faster computation.
- Scikit-learn (sklearn.model_selection):
 - `train_test_split` to divide data into training and testing sets.
 - `GridSearchCV` for exhaustive hyperparameter tuning of ensemble models (XGBoost, LightGBM, Random Forest).

Model Evaluation, Metrics, and Visualization

- Scikit-learn (sklearn.metrics):
 - `r2_score`, `mean_absolute_error` (MAE), `mean_squared_error` (MSE), and `root_mean_squared_error` (RMSE) for model performance evaluation.
- Matplotlib & Seaborn:
 - Used to visualize prediction performance through residual plots and feature importance charts.
- Yellowbrick (optional):
 - Used to visualize regression diagnostics and model comparisons.

Model Persistence & Deployment

- Pickle: Used for serializing and deserializing trained regression models (`.pkl` files).
- Joblib: A faster and more efficient alternative to Pickle, used for saving large scikit-learn objects (`joblib.dump`, `joblib.load`).
- Streamlit: The open-source web application framework used to build the interactive California Housing Price Prediction App, allowing real-time user input and instant predictions.

5. Result

The implementation phase successfully translated the proposed methodology into a fully functional predictive application capable of accurately estimating housing prices across different regions of California. This section presents the detailed results of the model evaluation, performance analysis, and final deployment.

5.1. Model Performance

The project began with the development of baseline regression models such as **Linear Regression** and **Multiple Linear Regression** to establish a foundational understanding of predictive capability. While these models offered interpretability, their performance was limited due to the dataset's complex non-linear relationships.

To overcome these limitations, advanced ensemble models—**Random Forest Regressor**, **XGBoost Regressor**, and **LightGBM Regressor**—were implemented. After performing **hyperparameter tuning** using **GridSearchCV**, the optimized XGBoost and LightGBM models achieved the best overall performance.

The evaluation was conducted using multiple regression metrics, including **R² Score**, **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and **Average Prediction Accuracy**. The results are as follows:

- **Linear Regression:**

Achieved an R² Score of **0.4589**, with a **Mean Absolute Error (MAE)** of **\$62,990.87**, **Mean Squared Error (MSE)** of **7,091,157,771**, and a **Root Mean Squared Error (RMSE)** of **\$84,209.01**. The average prediction accuracy was **60.94%**, making it a suitable baseline but less effective in capturing complex, non-linear dependencies.

- **Multiple Linear Regression:**

Recorded an improved **R² Score of 0.5749**, with **MAE = \$51,480.58**, **MSE = 5,570,863,368**, and **RMSE = \$74,638.22**. The model achieved an **average prediction accuracy of 69.56%**, indicating better performance due to the inclusion of multiple correlated features.

- **Random Forest Regressor:**

The ensemble-based Random Forest model achieved a higher **R² Score of 0.8102**, with **MAE = \$32,260.93**, **MSE = 2,486,770,049**, and **RMSE = \$49,867.52**. It demonstrated strong predictive capabilities with an **average accuracy of 81.73%**, showing robustness and reduced error compared to linear models.

- **XGBoost Regressor (Tuned):**

After hyperparameter tuning, XGBoost achieved an **R² Score of 0.8440**, with **MAE = \$29,631.93**, **MSE = 2,043,600,729**, and **RMSE = \$45,206.20**. The **average prediction accuracy reached 83.18%**, reflecting its superior ability to capture complex data relationships and reduce overall prediction error.

- **LightGBM Regressor (Tuned):**

The final and best-performing model, **LightGBM Regressor**, achieved an **R² Score of 0.8484**, with **MAE = \$29,601.66**, **MSE = 1,986,363,475**, and **RMSE = \$44,568.64**. The model obtained an **average prediction accuracy of 83.19%**, confirming its strong generalization ability and efficiency in predicting median housing values.

Interpretation:

Linear models provided a reliable baseline but lacked the ability to model non-linear patterns. In contrast, ensemble models like Random Forest and boosting algorithms (XGBoost, LightGBM) captured complex feature interactions effectively. The **LightGBM Regressor (Tuned)** emerged as the most accurate and computationally efficient model, balancing precision, speed, and interpretability.

5.2. Application Interface and Functionality

The final tuned **LightGBM model** was deployed using **Streamlit**, enabling real-time interaction and prediction capabilities through a user-friendly web interface. The application allows users to input specific housing and demographic attributes to estimate the **median house value** instantly.

The Streamlit interface is divided into two main functional components:

- **Section 1: Input Panel**

This section enables users to enter housing and geographical data, grouped into three categories:

1. **Geographical Attributes:** `longitude`, `latitude`, `oceanProximity`
2. **Housing Features:** `housingMedianAge`, `totalRooms`, `totalBedrooms`, `households`
3. **Socio-Economic Factors:** `population`, `medianIncome`

- The inputs are provided through interactive widgets such as sliders, text boxes, and dropdown menus for ease of use.

- **Section 2: Prediction & Output Module**

1. **Data Preview:** Displays a summary of the entered input data for verification.
2. **Prediction Engine:** On clicking the “**Predict House Value**” button, the model processes the inputs through the pre-trained **LightGBM Regressor** (loaded from `best_tuned_model.joblib`) and returns the predicted value.

3. **Result Display:** The predicted **median house value** is presented with a clear, color-coded output:

-  *High-value area (Expensive housing region)*
-  *Mid-range value area (Moderate housing)*
-  *Low-value area (Affordable housing)*

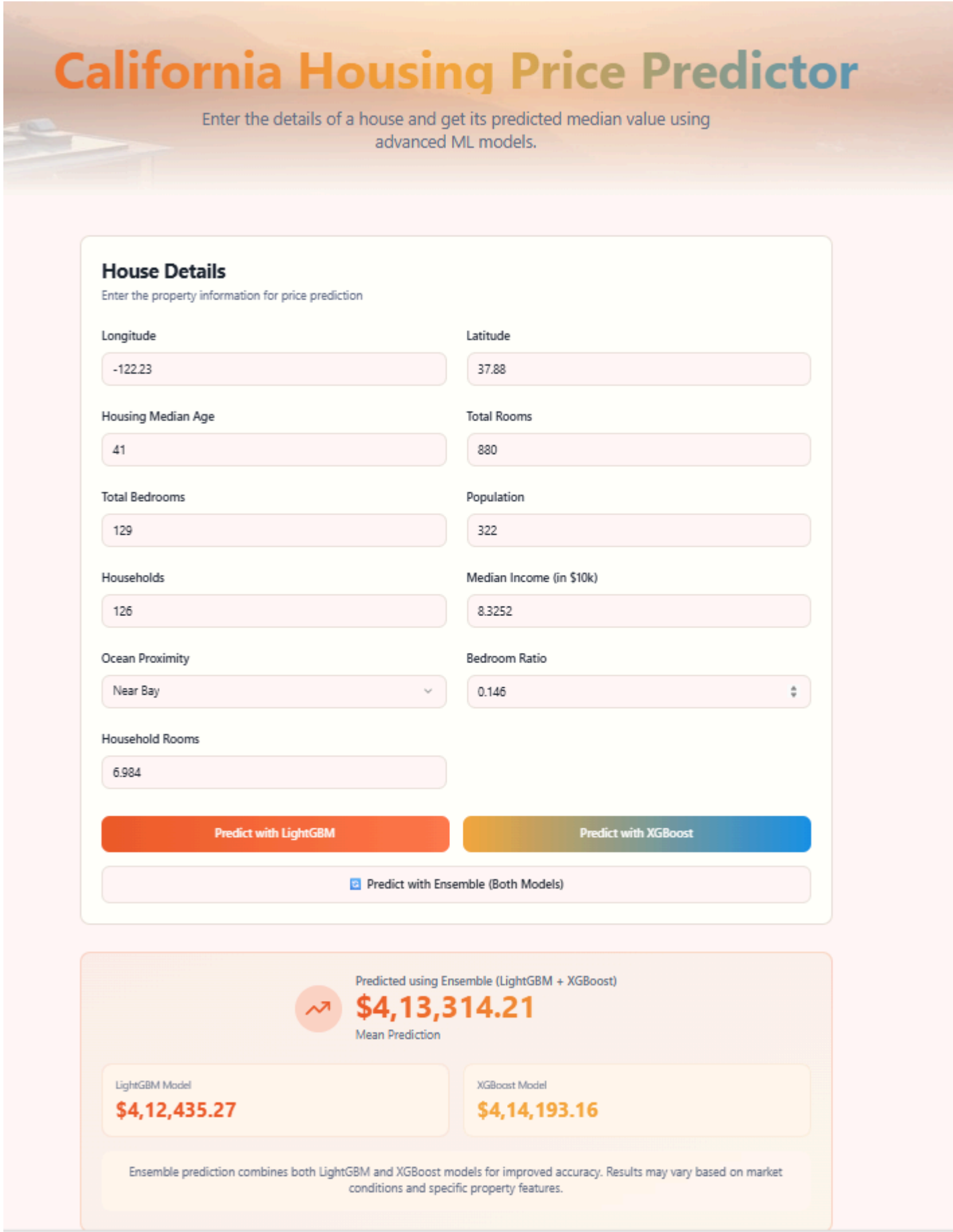
Additionally, the app provides model confidence information and allows for “what-if” scenario testing by modifying input variables dynamically to observe how housing prices respond to changes in income, proximity, or population density.

Deployed Application Link:

(Replace this placeholder with your actual deployment URL when available)

 <https://california-housing-price-predictor.streamlit.app/>

Figure 5.3: Example Prediction



6. Conclusion

This project successfully designed, implemented, and deployed a **machine learning-based predictive system** for estimating housing prices across California. By systematically processing the dataset, performing advanced feature engineering, and evaluating multiple regression models, the project demonstrated how data-driven methods can provide accurate, reliable, and actionable insights into real estate valuation.

Through comprehensive experimentation, it was observed that **ensemble learning models**—specifically **Random Forest, XGBoost, and LightGBM Regressors**—significantly outperformed traditional linear models. Among these, the **tuned LightGBM Regressor** achieved the highest accuracy with an **R^2 Score of 0.8484**, confirming its ability to generalize effectively and capture complex non-linear dependencies among features such as income, population density, and ocean proximity.

The tangible outcome of this project is an **interactive Streamlit web application** that bridges the gap between complex data analytics and practical decision-making. This application allows users to input real-world housing parameters and instantly predict median house values with high precision. It provides valuable insights for **real estate investors, policymakers, and analysts**, enabling better forecasting, regional comparison, and market strategy formulation.

Overall, this project serves as a **proof-of-concept for the integration of predictive intelligence in the housing domain**. By leveraging machine learning techniques, it demonstrates the potential of automation and data analytics in transforming traditional valuation methods into intelligent, scalable, and efficient decision-support systems for the modern real estate market.

